VNU-HCM UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

INTRODUCTION TO ARTIFICIAL INTELLIGENCE

PROJECT 1 - SEARCH

INSTRUCTORS:

Dr Nguyễn Tiến Huy
Mr Nguyễn Trần Duy Minh
Mr Nguyễn Thanh Tình

MEMBERS:

Lê Nguyên Thảo - 23127118
Đỗ Đăng Nhật Tiến - 23127269
Nguyễn Thanh Owen - 23127447
Trần Minh Quang - 23127464

Table of contents

1 Level 1: Blue Ghost using BFS **5**
1.1 Explanation of Searching Algorithm . . . 5
1.1.1 Pathfinding with BFS . . . 5
1.1.2 Updating and Moving the Blue Ghost . . . 5
1.1.3 Collision Avoidance Between Ghosts . . . 6
1.1.4 Summary of the Algorithm Workflow . . . 6
1.2 Test case . . . 7

2 Level 2: Pink Ghost using DFS **7**
2.1 Explanation of Searching Algorithm . . . 7
2.1.1 Pathfinding with DFS . . . 7
2.1.2 Updating and Moving the Blue Ghost . . . 8
2.1.3 Collision Avoidance Between Ghosts . . . 8
2.1.4 Summary of the Algorithm Workflow . . . 8
2.2 Test case . . . 9

3 Level 3: Orange Ghost using UCS **9**
3.1 Explanation of Searching Algorithm . . . 9
3.1.1 Pathfinding with UCS . . . 9
3.1.2 Cost Calculation . . . 10
3.1.3 Updating and Moving the Ghost . . . 10
3.1.4 Collision Avoidance Between Ghosts . . . 11
3.1.5 Summary of the Algorithm Workflow . . . 11
3.2 Test case . . . 11

4 Level 4: Red Ghost using A* Search **12**
4.1 Explanation of Searching Algorithm . . . 12
4.1.1 Pathfinding with A* Search . . . 12
4.1.2 Cost Calculation in A* Search . . . 13
4.1.3 Updating and Moving The Ghost . . . 14
4.1.4 Collision Avoidance Between Ghosts . . . 14
4.1.5 Summary of the Algorithm Workflow . . . 15
4.2 Test case . . . 15

5 Level 5: Parallel execution **15**
5.1 Overview . . . 15
5.2 Ghost Movement and Path finding Algorithms . . . 15
5.3 Parallel Execution Mechanism . . . 16

6 User-Controlled Pac-Man **16**
6.1 Game Mechanism . . . 16
6.2 Ghost Behavior & Path finding . . . 16
6.3 Ghost Movement Strategy . . . 17
6.4 Special Handling for the Pink Ghost . . . 17

7 Performance Evaluation of Path-finding Algorithms **17**

8 Demo Video **20**

9 Reference **20**

Member information

Full Name	**Student ID**	**Email**
Lê Nguyên Thảo	23127118	lnthao23@clc.fitus.edu.vn
Đỗ Đăng Nhật Tiến	23127269	ddntien23@clc.fitus.edu.vn
Nguyễn Thanh Owen	23127447	ntowen23@clc.fitus.edu.vn
Trần Minh Quang	23127464	tmquang23@clc.fitus.edu.vn

Table 1: List of member

Task and Completion

Student ID	**Name**	**Task**	**Percent**
23127118	Lê Nguyên Thảo	Level 2, 6 and Report	100%
23127269	Đỗ Đăng Nhật Tiến	Level 4, 5 and Report	100%
23127447	Nguyễn Thanh Owen	Level 1, 6 and Report	100%
23127464	Trần Minh Quang	Level 3, 5 and Video Demo	100%

Table 2: Assignment

1 Level 1: Blue Ghost using BFS

1.1 Explanation of Searching Algorithm

1.1.1 Pathfinding with BFS

The game map is represented as a 2D matrix, where each cell corresponds to a coordinate. The blue ghost uses **Breadth-First Search (BFS)** to find the shortest path to Pac-Man. Importantly, **after moving one tile (one step), the ghost recalculates its path** to ensure it continuously follows Pac-Man's latest position.

Step 1: Initialization

- Initialize the queue (**frontier**): The queue stores the positions to be explored, with the blue ghost's starting position pushed into the queue.
- Use a dictionary (**parent**) to store parent-child relationships: This helps reconstruct the path once the target is found.
- Create a set (**explored**) to store already-visited positions to prevent redundant checks.

Step 2: BFS Traversal

- Dequeue a position from **frontier**.
- Check if it is the target (Pac-Man's position):
  - If yes, reconstruct the path by backtracking through **parent** and return it.
- If not, expand in four possible directions (**Right, Left, Up, Down**).
  - If the new position is not a wall, not visited, and not occupied by another ghost, add it to **frontier** and mark its parent in **parent**.

Step 3: Handling Failure Cases

- If BFS completes without finding a valid path, it returns an empty list ([]), meaning the ghost will remain idle or try recalculating the path.

1.1.2 Updating and Moving the Blue Ghost

Once BFS finds the shortest path (**blue_Path**), the blue ghost moves along that path. Since Pac-Man moves dynamically, the ghost **recalculates its path after moving one tile** to adapt to Pac-Man's new position.

Step 1: Checking the Path

- If **blue_Path** is empty, the ghost runs BFS again to find a new path.
- If **blue_Path** is not empty, it checks if the ghost has reached the next target position:
  - If yes, recalculate the path using BFS from the new position to Pac-Man's updated location.
  - If no valid path exists, the ghost remains in place.

Step 2: Updating the Target

- If **blue_Path** contains a valid route, the next target position (**self.target**) is updated.
- The global list **ghost_targets** is also updated with this target, ensuring other ghosts do not collide with this position.

Step 3: Moving the Ghost

- If the x-coordinate is already aligned, the ghost moves vertically toward the target.
- Otherwise, it moves horizontally to reach the target.
- **After moving one tile, BFS is executed again** to update the path dynamically.

1.1.3 Collision Avoidance Between Ghosts

The ghost avoids collisions with other ghosts using the following approach:

- Uses a shared list (**ghost_targets**) to track the destinations of all ghosts.
- Avoids moving into a position that another ghost is targeting.
- If no valid move is available, the ghost remains in place for that frame.

1.1.4 Summary of the Algorithm Workflow

- BFS finds the shortest path to Pac-Man.
- The ghost updates its path and sets a movement target.
- The ghost moves step by step towards the target.
- Collision avoidance is managed using **ghost_targets**.
- The map will be a 2D matrix where each cell represents a tile. After moving one tile, the ghost recalculates and updates its path dynamically to follow Pac-Man moves.

1.2 Test case

Test times	Pac-Man's position	Search time (sec)	Memory usage (kb)	Number of expanded nodes
1st	Center of the map	0.0030	40.0	232
2nd	Top-left	0.0030	42.0	266
3rd	Top-right	0.0030	42.5	270
4th	Bottom-left	0.0040	77.6	312
5th	Bottom-right	0.0040	77.6	314

Table 3: Test cases for BFS Search

2 Level 2: Pink Ghost using DFS

2.1 Explanation of Searching Algorithm

2.1.1 Pathfinding with DFS

The Pink Ghost utilizes the **Depth-First Search (DFS)** algorithm to determine its path towards Pac-Man. A key difference is that the Pink Ghost follows the entire computed path before recalculating, instead of recalculating after each step like BFS.

Step 1: Initialization

- Stack-based frontier **(stack)**: A stack data structure is used to facilitate the depth-first exploration, with the starting position of the Pink Ghost being pushed onto the stack.
- Parent dictionary **(parent)**: Stores the parent-child relationship for path reconstruction once the target is found.
- Explored set **(explored)**: Keeps track of visited positions to avoid redundant checks.

Step 2: DFS Traversal

- Pop a position from the stack.
  - If it matches, the path is reconstructed using backtracking from the parent dictionary.
- Otherwise, it expands in four possible directions **(Right, Left, Up, Down)**.
  - If the new position is not a wall, not visited, and not occupied by another ghost, it is pushed onto the stack and recorded in the parent dictionary.

Step 3: Handling Failure Cases

- If DFS completes without finding a valid path, it returns an empty list (*[]*), causing the Pink Ghost to remain idle or attempt another recalculation in the next iteration.

2.1.2 Updating and Moving the Blue Ghost

Once DFS determines a path (**pink_Path**), the Pink Ghost follows it entirely before recalculating.

Step 1: Checking the Path

- If **pink_Path** is empty, the ghost runs DFS to find a new path.
- If **pink_Path** is not empty, the ghost moves to the next position on the path.
- If the ghost reaches the target, it recalculates its path.

Step 2: Moving the Ghost

- The ghost moves horizontally or vertically toward the next position in **pink_Path**.
- The global list **reversed_Positions** updates to track ghost positions and prevent collisions.
- The Pink Ghost follows this computed path until it reaches the last tile before recalculating.

2.1.3 Collision Avoidance Between Ghosts

To prevent multiple ghosts from occupying the same position:

- A shared list **(reversed_Positions)** tracks the destinations of all ghosts.
- The Pink Ghost avoids moving into a position occupied by another ghost.
- If no valid move exists, the ghost remains in place for that frame.

2.1.4 Summary of the Algorithm Workflow

- DFS explores potential paths to Pac-Man.
- The Pink Ghost follows the entire computed path before recalculating.
- The ghost moves step by step towards the target.
- Collision avoidance is managed through **reversed_Positions**.
- DFS recalculates the path only after reaching the computed target.

2.2 Test case

Test times	Pac-Man's position	Search time (sec)	Memory usage (kb)	Number of expanded nodes
1st	Center of the map	0.003	48.1	274
2nd	Top-left	0.0010	22.6	79
3rd	Top-right	0.0020	36.8	170
4th	Bottom-left	0.0035	48.1	268
5th	Bottom-right	0.0020	37.5	185

Table 4: Test cases for DFS Search

3 Level 3: Orange Ghost using UCS

3.1 Explanation of Searching Algorithm

3.1.1 Pathfinding with UCS

Uniform Cost Search (UCS) is a graph search algorithm that finds the least-cost path from a start node to a target node by expanding the lowest-cost node first. It operates using a priority queue and follows these steps:

Step 1: Data Structures Initialization

- The algorithm begins at a given start position.
- A **priority queue** is initialized with the start position and a cost of *0*.
- A **parent dictionary** keeps track of the path.
- A **cost dictionary** stores the cumulative cost of reaching each node.
- A **set of explored nodes** keeps track of visited positions.
- The algorithm considers movement in four directions (right, left, up, down).

Step 2: Lowest-Cost Node Expansion

- The node with the **lowest cost** is extracted from the priority queue.
- If the node matches the target position, the algorithm **reconstructs the path** by backtracking through the parent dictionary.
- If not, the node is marked as explored.

Step 3: Generate Successors

- The algorithm evaluates all possible moves from the current node.
- If a move leads to a valid and traversable position (i.e., it's not a wall or an obstacle), it calculates the new cost.

Step 4: Priority Queue Update

If the new position has not been visited before or a lower-cost path is found, the algorithm:

- Updates the cost dictionary.
- Stores the parent node for path reconstruction.
- Adds the new position to the priority queue with its cost.

Step 5: Repetition Until Target Achievement

- The process continues until the target is found or the priority queue is empty.
- If the queue is empty before reaching the target, no path exists.

3.1.2 Cost Calculation

The movement cost from one grid cell to another is calculated as:

$$move_cost = |cost[new_pos] - cost[current]| + 1$$

- *cost[new_pos]* and *cost[current]* are values from a predefined cost matrix.
- The **absolute difference** between these values ensures that UCS prefers lower-cost paths.
- *+1* ensures that every movement has a **positive** cost.

Example:

- If moving from a cell with **cost = 3** to one with **cost = 5**, the move cost is:

$$|5 - 3| + 1 = 3$$

- This means moving between different terrain types (e.g., road vs. grass) can have different costs.

3.1.3 Updating and Moving the Ghost

The orange ghost continuously updates its movement to follow the shortest path to its target while adapting to dynamic changes in the game environment.

Step 1: Target and Path Determination

- The ghost ensures its target aligns with the grid to maintain smooth movement.
- If no path is currently assigned, it calculates the shortest route using Uniform Cost Search (UCS).
- After obtaining the path, the first position is removed to prevent unnecessary reprocessing.

Step 2: Efficient Path Following

- As the ghost moves, it checks if it has reached the next position in its path.
- If so, it removes that position from the path and recalculates using UCS to ensure an optimal route.
- This allows the ghost to dynamically adjust if obstacles appear or the environment changes.

Step 3: Movement Execution

- The ghost moves one step at a time toward its next target.
- If the target is directly above or below, it moves vertically (up/down).
- If the target is to the left or right, it moves horizontally (left/right).
- The movement speed remains constant for a smooth and predictable path.

3.1.4 Collision Avoidance Between Ghosts

To prevent multiple ghosts from colliding or overlapping, a few strategies are applied:

- The orange ghost is aware of other ghosts' positions through a shared list (ghost_targets).
- Before moving, it checks if another ghost is occupying or moving toward the same grid space.
- It avoids moving into the other ghosts' paths.
- If there's not any move available, the ghost stays at its current position for that frame.

3.1.5 Summary of the Algorithm Workflow

- UCS calculates the shortest path to the target,
- The ghost sets a movement target and moves step by step towards the target.
- The ghost avoids collisions by using a shared list (ghost_targets).
- The map will be a 2D matrix where each cell represents a tile. After moving one tile, the ghost recalculates and updates its path dynamically to follow Pac-Man moves.

3.2 Test case

Test times	Pac-Man's position	Search time (sec)	Memory usage (kb)	Number of expanded nodes
1st	Center of the map	0.0030	49.4	224
2nd	Top-left	0.0040	52.1	266
3rd	Top-right	0.0040	53.2	288
4th	Bottom-left	0.0040	87.0	313
5th	Bottom-right	0.0050	87.0	327

Table 5: Test cases for UCS Search

4 Level 4: Red Ghost using A* Search

4.1 Explanation of Searching Algorithm

4.1.1 Pathfinding with A* Search

A* is an informed search algorithm that finds the shortest path from a start position to a target position while considering both cost and estimated distance. It uses a combination of actual cost (**g-cost**) and heuristic cost (**h-cost**) to prioritize the most promising paths.

Step 1: Initialization

- The algorithm starts by identifying the **initial position**.
- A **priority queue** is initialized, storing nodes sorted by their total cost (**f-cost = g-cost + h-cost**).
- A **parent dictionary** is used to reconstruct the path once the target is reached.
- A **g-cost dictionary** tracks the shortest known cost to reach each node.
- A **set of explored nodes** prevents redundant calculations.

Step 2: Best Node Extraction

- The algorithm selects the node with the **lowest f-cost** from the priority queue (i.e., the most promising path).
- If the selected node is the **target**, the algorithm terminates, and the shortest path is reconstructed.
- Otherwise, the node is added to the **explored set**, marking it as visited.

Step 3: Neighboring Node Generation

- The algorithm evaluates **all possible movement directions** (right, left, up, down).
- The **new position** is calculated by applying these movement directions.
- Before processing a new position, it checks:
  - If it has already been explored.
  - If it is within the grid boundaries and not an obstacle.
  - If it belongs to **a valid walkable tile** (e.g., open paths, non-blocking zones).

Step 4: Neighboring Node Cost Calculation

- **g-cost**: This is the actual movement cost from the start to the current node.
- **h-cost (heuristic)**: This is the estimated cost from the current position to the target
- **f-cost** (h-cost + g-cost): The total cost of the node

Step 5: Best Path Update

If the new path to a node is **cheaper** than a previously recorded path, the algorithm updates:

- The **g-cost dictionary** with the lower cost.
- The **priority queue**, ensuring the new node is processed efficiently.
- The **parent dictionary**, allowing backtracking to reconstruct the path.

Step 6: Iteration Until Target Discovery

Steps 2 to 5 repeat until:

- The target position is reached, in which case the algorithm reconstructs the shortest path.
- The priority queue becomes empty, meaning no valid path exists.

Step 7: Path Reconstruction

- If the target is found, the algorithm **traces back** from the target to the start using the **parent dictionary**.
- The reconstructed path is then reversed to guide movement step by step from the start to the target.

Step 8: No Path Situation Handling

If no valid path is found, the algorithm returns an empty path, indicating the target is unreachable due to obstacles.

4.1.2 Cost Calculation in A* Search

- **Calculate Movement Cost (g-cost)**
  The movement cost from one grid cell to another is calculated as:

$$g(n) = move_cost = |cost[new_pos] - cost[current]| + 1$$

  - *cost[new_pos]* and *cost[current]* are values from a predefined cost matrix.
  - The **absolute difference** between these values ensures that UCS prefers lower-cost paths.
  - *+1* ensures that every movement has a **positive** cost.

- **Calculate Heuristic Cost (h-cost)**
  The heuristic function used is the **Manhattan Distance**, which is calculated as:

$$h(n) = |x_{goal} - x_n| + |y_{goal} - y_n|$$

  - It estimates the remaining steps to the target assuming no obstacles.
  - Works well for **grid-based movement** (right, left, up, down).

- Calculate Total Cost (f-cost)

$$f(n) = g(n) + h(n)$$

  - *g(n)*: Cost from start to n (path cost).
  - *h(n)*: Estimated cost from n to goal.

4.1.3 Updating and Moving The Ghost

Step 1: Setting the Target Position

- The ghost determines its **target** (usually the Pac-Man's position).

Step 2: Path Calculation

- The ghost calculates a **shortest path** using A* search, which considers both the actual travel cost (g-cost) and an estimated heuristic (h-cost).
- If a path does not exist or needs to be updated, a new A* search is performed.

Step 3: Target-Oriented Movement

- The ghost follows the computed path **step by step**.
- Movement is restricted to four possible directions: **up, down, left, or right**.
- At each step, the ghost moves toward the next position along the path.

Step 4: Continuous Updates

- The path is **continuously recalculated** as the player moves.
- If the ghost reaches its current target position, the next waypoint in the path is retrieved.

4.1.4 Collision Avoidance Between Ghosts

To prevent multiple ghosts from colliding or overlapping, a few strategies are applied:

- It follows the same principles as the other ghosts to ensure smooth movement and avoid collisions.
- The red ghost is aware of other ghosts' positions through a shared list (**ghost_targets**).
- Before moving, it checks if another ghost is occupying or moving toward the same grid space.
- It avoids moving into the other ghosts' paths.
- If there's no available move, the red ghost stays at its current position for that frame.

4.1.5 Summary of the Algorithm Workflow

- Set the target position (typically the player's location).
- A* search compute the shortest path.
- Follow the calculated path by moving **step-by-step**.
- The map will be a 2D matrix where each cell represents a tile. After moving one tile, the ghost recalculates and updates its path dynamically to follow Pac-Man moves.
- Avoid collisions by considering other ghosts' positions.

4.2 Test case

Test times	Pac-Man's position	Search time (sec)	Memory usage (kb)	Number of expanded nodes
1st	Center of the map	0.0010	12.3	48
2nd	Top-left	0.0010	11.3	33
3rd	Top-right	0.0010	9.2	31
4th	Bottom-left	0.0010	13.5	53
5th	Bottom-right	0.0020	14.0	50

Table 6: Test cases for A* Search

5 Level 5: Parallel execution

5.1 Overview

- At this level, we implement simultaneous movement for all four ghosts (Blue, Pink, Orange, and Red) in the same maze. Each ghost follows its respective search algorithm to chase Pac-Man while ensuring that they do not collide with each other.

5.2 Ghost Movement and Path finding Algorithms

Each ghost employs a different search algorithm to determine its path to chase Pac-Man:

- Blue Ghost (BFS): Uses Breadth-First Search to find the shortest unweighted path.
- Pink Ghost (DFS): Uses Depth-First Search, prioritizing deeper paths before backtracking.
- Orange Ghost (UCS): Uses Uniform-Cost Search, considering the cost of movement across different paths.
- Red Ghost (A* Search): Uses A Search, incorporating both path cost and heuristic distance to optimize movement.

5.3 Parallel Execution Mechanism

To ensure that all four ghosts move simultaneously without collisions, the following mechanisms are applied:

- **Independent path finding:** Each ghost calculates its own path of movement based on Pac-Man's position and its assigned path finding algorithm. All ghosts determine their next move before executing their movement.

- **Recalculating Path After Every Move:** After computing a path, each ghost selects its next tile as the destination and recalculates its path after moving one step.

- **Collision avoidance using ghost_targets array:**

- A **ghost_targets** array stores the intended destination of each ghost.
- Ghosts treat positions in **ghost_targets** as walls, preventing them from moving into those tiles.
- If a ghost cannot move because all valid directions are occupied, it remains in its current position for that frame.

6 User-Controlled Pac-Man

6.1 Game Mechanism

- Pac-Man's movement is controlled using the arrow keys on the keyboard: **right, left, up, down.**

- After each key press, the program detects the input and updates Pac-Man's direction accordingly. If Pac-Man is able to turn based on his current position, it will follow the new direction. However, if the chosen direction is blocked (e.g., a wall), Pac-Man will continue moving in its current direction until it reaches a valid turning point.

- The player's score increases as Pac-Man consumes the dots on the map:

- Small dots: **10 points**
- Large dots: **50 points**

- When Pac-Man eats a **large dot**, all ghosts stop moving for **3 seconds** before resuming their chase based on their respective **path finding algorithms**. If Pac-Man collides with a ghost, it will respawn at the starting position, and all ghosts will also return to their initial positions before continuing their pursuit.

- Each player has **three lives**. If all lives are lost, pressing the `Space` key restarts the game from the beginning. The game ends when

- **Victory**: Pac-Man eats all dots → **green message** appears.
- **Defeat**: Pac-Man loses all lives → **red message** appears.

6.2 Ghost Behavior & Path finding

- This behavior is similar to **Level 5**. Each ghost is assigned a specific path-finding algorithm:

- **Blue Ghost**: Breadth-First Search (BFS)

- **Pink Ghost**: Depth-First Search (DFS)
- **Orange Ghost**: Uniform Cost Search (UCS)
- **Red Ghost**: A* Search Algorithm

- The game map is represented as a **2D matrix**, where both Pac-Man and ghosts move one tile at a time. Every time a ghost moves **one tile**, it recalculates the shortest path to Pac-Man using its assigned path-finding algorithm to ensure continuous pursuit.

6.3 Ghost Movement Strategy

- Each ghost has two primary objectives:

1. **Chasing Pac-Man** using its predefined search algorithm.
2. **Avoiding collisions with other ghosts** by considering their predicted movements.

- To prevent ghosts from colliding, the program maintains an **array storing the next steps of all ghosts**. Each ghost treats the predicted positions of the other ghosts as obstacles and avoids moving into them. This ensures that the ghosts never overlap during the chase.

6.4 Special Handling for the Pink Ghost

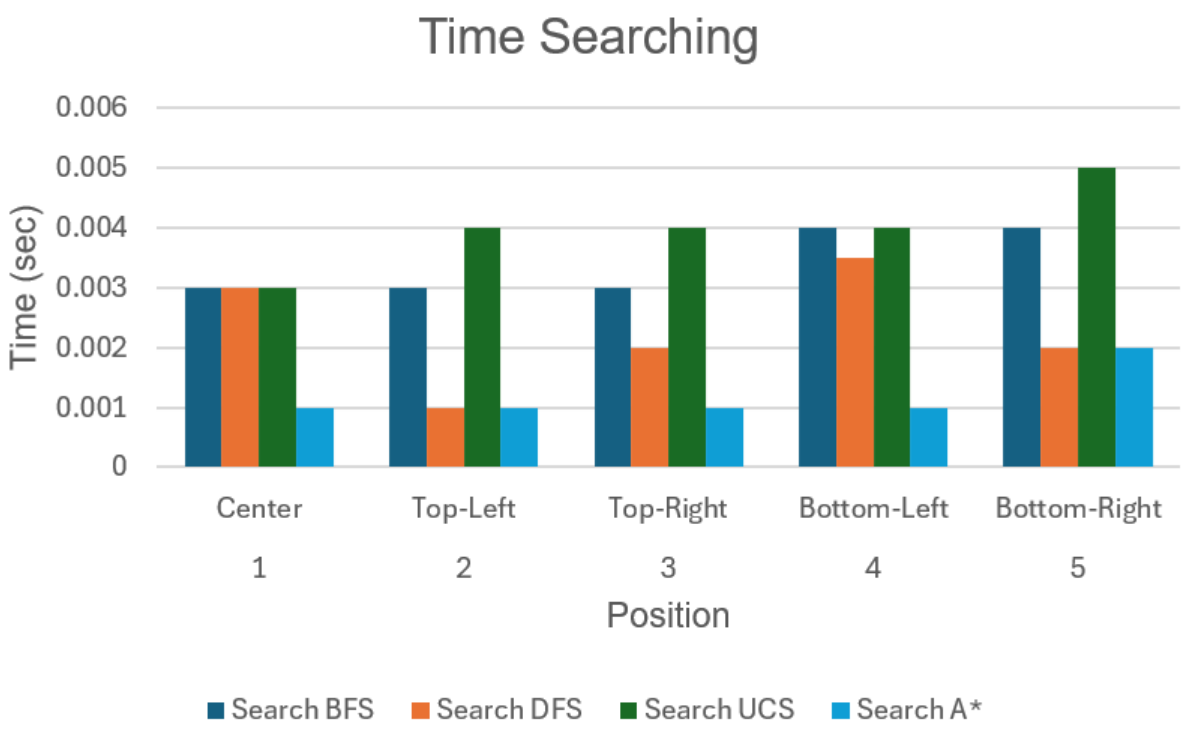
- The Pink Ghost has a unique behavior issue where it may repeatedly move back and forth at a single position. To resolve this, the Pink Ghost follows these steps:

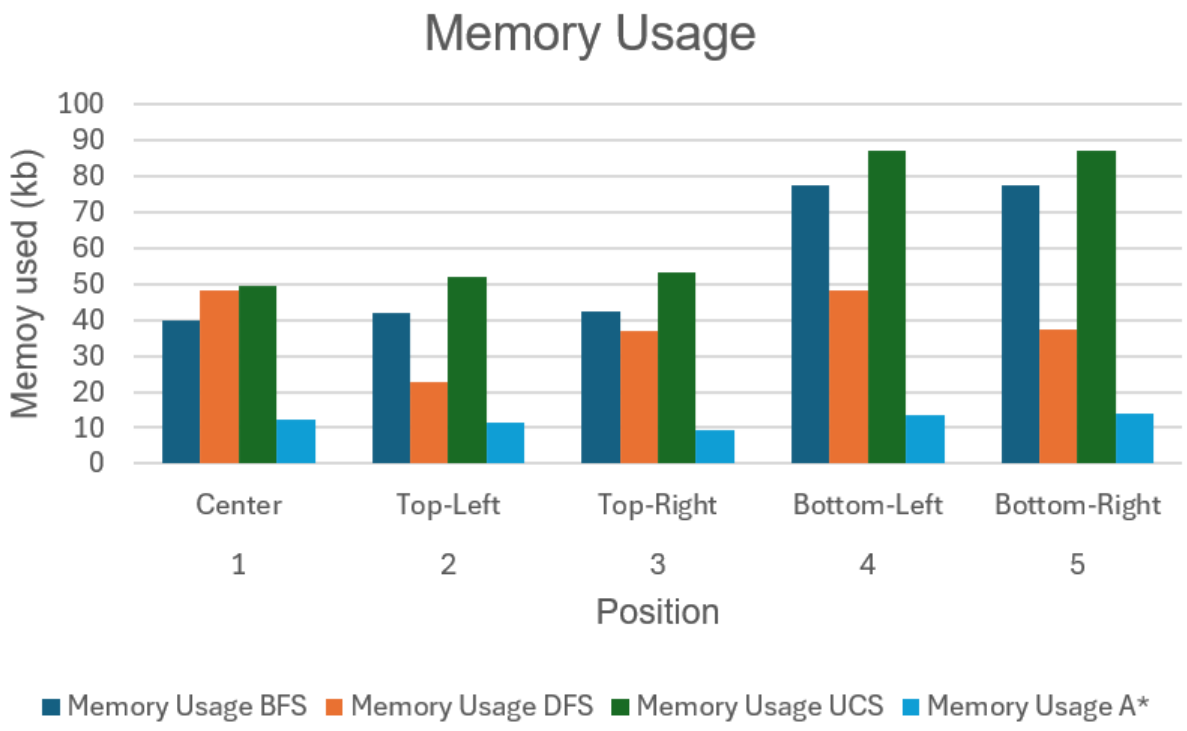
- It first moves towards a specific location of Pac-Man that it has previously identified.
- Once it reaches that location, the it recalculates a new path to Pac-Man's latest position.

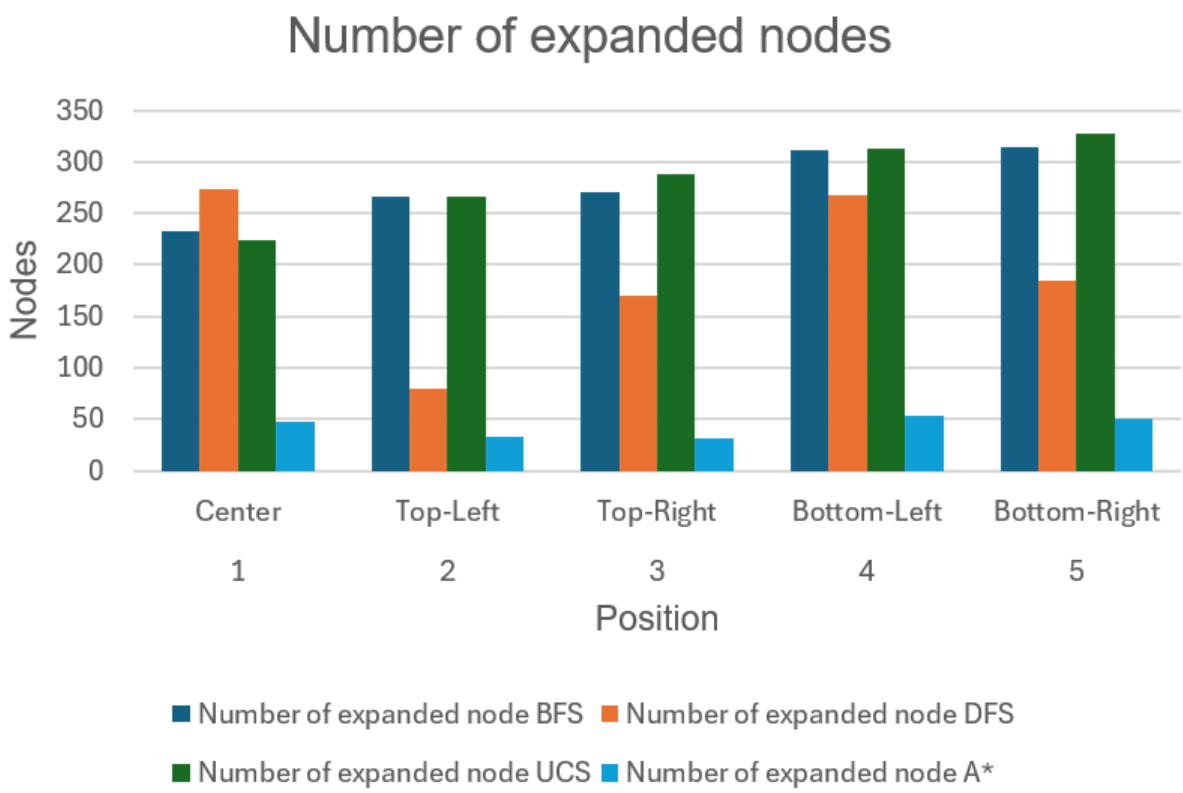
- This will help the Pink Ghost to remain engaged in pursuing Pac-Man without getting stuck in repetitive loops.

7 Performance Evaluation of Path-finding Algorithms

- From **the test cases of level 1, 2, 3, 4**. We have the following charts:


Time Searching
Time (sec)
0.006
0.005
0.004
0.003
0.002
0.001
0
Center
Top-Left
Top-Right
Bottom-Left
Bottom-Right
1
2
3
4
5
Position
Search BFS
Search DFS
Search UCS
Search A*


Memory Usage
Memoy used (kb)
100
90
80
70
60
50
40
30
20
10
0
Center
Top-Left
Top-Right
Bottom-Left
Bottom-Right
1
2
3
4
5
Position
Memory Usage BFS
Memory Usage DFS
Memory Usage UCS
Memory Usage A*



Based on the evaluation charts, the performance of BFS, DFS, UCS, and A* is analyzed in terms of three key factors:

- **Search Time** – The duration required to locate a path to Pac-Man.
- **Memory Usage** – The peak amount of memory consumed while storing explored nodes.
- **Number of Expanded Nodes** – The total number of nodes visited before reaching the goal.

The following table summarizes the findings from the visualized data:

Algorithm	Search Time	Memory Usage	The Number Of Expanded Nodes
BFS	Moderate	High	High
DFS	Unstable	Low	Low but not optimal
UCS	Slowest	Highest	High
A*	Fastest	Lowest	Lowest

Table 7: Performance Comparison of Path-finding Algorithms

Conclusion:

- The analysis shows that **A*** consistently outperforms the other algorithms in terms of **search time**, **memory efficiency**, and **the number of expanded nodes**. **BFS** and **UCS** tend to explore a large number of nodes, leading to higher memory consumption and longer search times. **DFS**, while sometimes faster, lacks stability and can lead to inefficient paths. Therefore, **A*** is **the most optimal choice** for path-finding in this scenario.

8 Demo Video

Link Drive: ***Demo Drive***
Link Youtube: ***Demo Youtube***

9 Reference

1. How to make Pac-Man in Python
2. Pygame and how to install pygame
3. BFS for Blue Ghost
4. DFS for Pink Ghost
5. UCS for Orange Ghost
6. A for Red Ghost*